

Informática General 2026

Cátedra: Valeria Drelichman, Pedro Paleo, Leonardo Nadel, Norma Morales

Alumnas: Rocío Binaghi / Micaela Rojas

Documentación TP N°1

1. Instrucción

El sitio Web cuenta con 6 páginas: el inicio que muestra los juegos que se van a ver, tres de juegos (cartas, dados y preguntas), de puntajes y la última que se encontraran datos del proyectos y sus participantes. Para programar se utilizó JavaScript.

Este trabajo se utilizó la herramienta de la Inteligencia Artificial como ChatGPT, Claude y Gemini.

El archivo está dividido en 6 archivos HTML y tres carpetas. Una carpeta de JavaScript que cuenta con 5 archivos js. Otra carpeta de CSS que cuenta con un archivo de estilos. Y por último una carpeta para las imágenes, que cuenta con imágenes y tres carpetas. Que para una mejor organización se guardaron en una de las carpetas las imágenes de cartas, en otra las imágenes de los dados y con las fotos de la alumnas.

2. Juegos

2.1 Juego de cartas

Casita robada

Este juego permite jugar de dos modalidades distintas: contra la compu o multijugador.

Instrucciones del juego:

Objetivo del Juego Capturar la mayor cantidad de cartas posibles de la mesa y de la pila ("casita") del rival para tener el pozo más grande al finalizar el mazo.

Preparación

- Se reparten **3 cartas** a cada jugador.
- Se colocan **4 cartas cara arriba** en el centro de la mesa.
- El resto de las cartas forman el mazo de reserva.

Mecánica del Turno En su turno, un jugador debe bajar **1 carta** de su mano y realizar una de estas acciones:

1. **Capturar de la mesa:** Si la carta que baja coincide en número/figura con alguna de la mesa, se lleva ambas cartas a su "casita" (su pila personal de captura).
2. **Robar la casita del rival:** Si la carta que baja coincide con la carta superior de la "casita" del oponente, se roba **toda la pila** del rival y la suma a la suya.
3. **Descartar:** Si no puede ni quiere capturar nada, deja su carta cara arriba en la mesa.

Flujo y Fin del Juego

- Cuando los jugadores se quedan sin cartas en la mano, se vuelven a repartir **3 cartas a cada uno** del mazo de reserva (sin volver a poner cartas en la mesa).
- El juego termina cuando se agotan todas las cartas del mazo.
- Gana el jugador que tenga la **"casita" con más cartas acumuladas**.

Interacción de usuario:

Al inicio la página muestra las instrucciones del juego y al finalizar hay un botón para iniciar partida. Una vez que se inicia la partida las instrucciones desaparecen pero hay un botón para verlas cuando sean necesarias.

1. **Instrucciones / Inicio:** Muestra `#seccionInstrucciones` y `#contenedorIniciar`.
2. **pasoJuego:** Oculta los formularios y despliega el tablero de juego activo (`.tablero-central`, `.mano-seccion`).

1° Captar datos

Primero Preguntarle si va a jugar con la computadora o con alguien más, se utilizan **botones para poder elegir**. Si llega a poner con la computadora se le va preguntar directamente su nombre con un input. Si coloca que juega con alguien, primero se le va preguntar con cuantos y luego el nombre de cada uno. Por cada participante que le pregunte el nombre aparecerá "*Nombre del participante :*" y el número de participante.

Configuración e Ingreso de Datos (Formularios y Eventos)

1. **Selección del modo:** Se registran eventos click en los botones iniciales (`#jugarContraCompu` y `#jugarConParticipantes`) para ocultar/mostrar los contenedores de configuración mediante el atributo hidden.
2. **Validación de datos:** Se capturan y validan las entradas del usuario (mínimo 2 jugadores en modo multijugador o nombre no vacío).
3. **Inicio del Tablero:** Tras ingresar todos los participantes, se habilita el contenedor principal del juego (`#pasoJuego`) y se invoca a `prepararPartida()`.

2° Iniciar el Juego

Se reparten 4 cartas en la mesa, y se inician los turnos. A cada jugador se le reparten tres cartas. Solo se van a ver las cartas del que es mano. en la pantalla se va a ver de quién es el turno. El usuario va a poder seleccionar una de sus cartas. Y decidirá si tirarla en la mesa mediante un **botón**, robar la carta del contrincante haciendo click en las cartas si son del mismo número. El juego termina cuando ya no haya más cartas en el mazo.

Interacciones con la mesa:

- **Jugar contra la Mesa (jugarContraMesa(posicionMesa)):** Verifica si **cartaMano.numero === cartaMesa.numero**. En caso afirmativo, mueve ambas cartas a la casita del jugador activo, actualiza **ultimoEnRobar** y pasa el turno.
- **Robar Casita Rival (robarCasita(posicionRival)):** Si el número de la carta seleccionada coincide con la carta superior de la casita de un contrincante, el jugador captura toda la pila del rival junto con la carta de su mano. Compara la carta seleccionada contra el tope de la casita rival (**rival.casita[rival.casita.length - 1]**). Si coinciden los números, transfiere la pila completa del rival mediante la sintaxis **spread ...rival.casita** más la carta de la mano a la casita propia del jugador.
- **Descartar a la Mesa (tirarAMesa):** Permite descartar la carta seleccionada enviándola directamente a la mesa. mediante un evento click sobre (**#tirarMesa**). Quita la carta seleccionada de la mano mediante **.splice()** y la agrega a la mesa.

Inicialización y Reparto de Cartas

- **crearMazo():** Genera las cartas mezclando los palos y los valores por palo, aplicando un ordenamiento aleatorio aplicando el orden aleatorio: **.sort(() => Math.random() - 0.5)**
- **prepararPartida():** Inicializa los objetos de cada jugador, dispone 4 cartas iniciales en la mesa e invoca a **repartirManos()**.
- **repartirManos():** Asigna 3 cartas del mazo a la mano de cada jugador.
- **crearElementoCarta(carta, accionAlClick):** Crea dinámicamente un contenedor **<div>** con una imagen cuya ruta depende del número y palo (**img/cartas/{numero}.{palo}.png**). Incluye un texto por si la imagen no se encuentra.

Ciclo del Turno

- **iniciarTurno():** Verifica si todos los jugadores se quedaron sin cartas. Si es así y queda mazo, reparte 3 más a cada uno; si no queda mazo, llama a **evaluarGanador()**. Si es el turno de la **Computadora**, se activa la bandera **bloqueado** y se ejecuta la lógica automatizada tras 1 segundo mediante **setTimeout()**.
- **Renderizado del DOM (actualizarPantalla()):** Limpia y vuelve a generar las cartas visuales en la mesa, la mano del jugador actual y las "casitas" acumuladas usando la función constructora **crearElementoCarta()**.

Jugar contra la computadora (turnosCPU)

Implementa una toma de decisiones por jerarquía de prioridades:

1° Robar Casita: Si encuentra alguna casita cuyo tope coincida con alguna carta ejecuta **robarCasita()**.

2° Capturar de Mesa: Si no puede robar casitas, evalúa si alguna de sus cartas en mano coincide con alguna carta libre de la mesa mediante un bucle anidado for. Si hay coincidencia, ejecuta **jugarContraMesa()**.

3° Descarte Forzado: Si no existen coincidencias, descarta automáticamente la primera carta de su mano (`mano.splice(0, 1)`) hacia la mesa.

3° Finalizar la partida

Una vez que no hay más cartas para repartir. La mesa de juego deja de ser visible. Aparecen los puntos. Un botón para iniciar nueva partida y otro botón para ir a la página de puntajes.

- **evaluarGanador():** Asigna las cartas restantes de la mesa al **ultimoEnRobar**. Calcula el total de cartas en cada casita y determina al ganador. Oculta el tablero de juego y muestra la pantalla/mensaje final con los resultados.
- **guardarPuntajesLocal():** Lee el historial desde el almacenamiento del navegador mediante **localStorage.getItem("casita_records")**. Convierte las cadenas a objetos con **JSON.parse()**, agrega la nueva partida, los ordena de mayor a menor y los vuelve a guardar serializados en formato **JSON.stringify()**.
- **Botón tablero de puntajes y Reinicio:** Permite reiniciar la partida limpia mediante **reiniciarJuego()** o redirigir a la tabla de posiciones con **window.location.href = "puntaje.html"** utilizando el objeto **location**.

Estructura de Datos y Estado Global

El estado del juego se controla a través de variables globales que mantienen la sincronización de la interfaz y la lógica:

- **esContraCompu** (*boolean*): Determina si el modo activo es contra la compu (true) o multijugador (false).
- **totalJugadores** (*number*): Cantidad de participantes seleccionados.
- **listaNombres** (*Array*): Almacena temporalmente los nombres ingresados.
- **jugadores** (*Array*): Contiene la lista de objetos de jugador con la estructura:
 - **nombre** (*string*)
 - **mano** (*Array*): Cartas actuales en posesión del jugador.
 - **casita** (*Array*): Pila acumulada de cartas robadas.
- **mazo** (*Array<Carta>*): Baraja actual ordenada aleatoriamente. Cada Carta es un objeto: { **numero**: *number*, **palo**: *string* }.
- **mesa** (*Array<Carta>*): Cartas visibles disponibles en el centro del tablero.
- **turnoActual** (*number*): Índice en **jugadores** que indica a quién le toca actuar.
- **cartaSeleccionada** (*number | null*): Índice de la carta seleccionada de la mano del jugador activo.
- **ultimoEnRobar** (*number | null*): Registra el índice del último jugador en realizar una captura para otorgarle las cartas remanentes al vaciar el mazo.

- **bloqueado (boolean):** Bandera que bloquea la interacción del usuario mientras la Computadora está procesando su turno.

2.2 Juego de Dados

"21 con Dados"

Este juego permite jugar de dos modalidades distintas: contra la compu o multijugador.

Instrucciones del juego:

Acumular la mayor cantidad de puntos posibles sin superar el límite de 21 en cada ronda. El primer usuario en ganar **3 rondas** se corona como el ganador definitivo de la partida.

Lanzamiento y Reglas de Puntuación

En cada tiro se lanzan 2 dados de 6 caras y el resultado modifica el puntaje acumulado de la ronda actual:

- **Suma Estándar:** El puntaje obtenido es la suma simple de ambos dados ($\$Dado_1 + Dado_2\$$).
- **Doble As (1 y 1):** Otorga la opción de sumar **2 o 14 puntos** (según le convenga al usuario para no pasarse).
- **Pares Iguales (2 y 2, 3 y 3, etc.):** Duplican la suma obtenida en esa tirada (por ejemplo, un doble 4 suma **16 puntos** en lugar de 8).

Mecánica del Turno

1. El usuario inicia su turno eligiendo entre **Tirar o Plantarse**.
2. Puede lanzar los dados cuantas veces desee mientras el total de la ronda no supere los 21 puntos.
3. **Pérdida por Exceso (Bust):** Si el acumulado supera los 21 puntos en cualquier tiro, el usuario suma **0 puntos** en esa ronda y su turno termina automáticamente.
4. **Plantarse:** Al presionar este botón, el usuario congela su puntaje actual y cede el turno al siguiente participante.

Sistema de Puntuación y Rondas

- **Puntos por Ronda:** Al finalizar los turnos de todos los participantes, se comparan las puntuaciones finales de la ronda.
- **Ganador de Ronda:** El usuario con la puntuación válida más alta (más cercana a 21) recibe **1 Victoria** en el marcador global.
- **Empate:** Si dos o más usuarios se plantan con la misma puntuación máxima, todos reciben **1 Victoria** en esa ronda.

- **Fin de la Partida:** La partida termina inmediatamente cuando un usuario alcanza un total de **3 Victorias**.

1° Captar datos

Primero Preguntarle si va a jugar con la computadora o con alguien más, se utilizan **botones para poder elegir**. Si llega a poner con la computadora se le va preguntar directamente su nombre con un input. Si coloca que juega con alguien, primero se le va preguntar con cuantos y luego el nombre de cada uno. Por cada participante que le pregunte el nombre aparecerá "*Nombre del participante :*" y el número de participante.

- 1. Selección del modo:** Se registran eventos click en los botones iniciales (`#jugarContraCompu` y `#jugarConParticipantes`) para ocultar/mostrar los contenedores de configuración mediante el atributo `hidden`.
- 2. Validación de datos:** Se capturan y validan las entradas del usuario (mínimo 2 jugadores en modo multijugador o nombre no vacío).
- 3. Inicio del Tablero:** Tras ingresar todos los participantes, se habilita el contenedor principal del juego (`#pasoJuego`) y se invoca a `iniciarEstructuraPartida()` y se Habilita los botones principales de interacción (`#tirarDados`, `#plantarse` y `#jugarDeNuevo`).

2° Iniciar el Juego

Se habilitan los botones Tirar dados, Plantarse y reiniciar partidas. El jugador va a tirar los dados hasta que llegue al número que desee o se va a plantar. Si se llega a pasar del 21 no se le toma ningún punto. Si gana la ronda se le suma una victoria. Luego de que algún participante llegue a tres victorias la partida finaliza. Se muestra la tabla de puntajes y el botón de reiniciar partida y ver tabla de puntajes.

Interacciones con la mesa:

- **Tirar Dados (`#tirarDados`):** Genera dos números aleatorios del 1 al 6 (`Math.floor(Math.random() * 6) + 1`). Si salen dos Ases (1 y 1), pausa el juego y despliega el modal emergente. De lo contrario, calcula el puntaje mediante `calcularPuntosTiro()` aplicando suma simple o duplicación de pares iguales, y ejecuta `aplicarPuntosTiro()`. Si el acumulado supera los 21 puntos, reinicia el puntaje a 0 y pasa el turno automáticamente.
- **Plantarse (`#plantarse`):** Permite al jugador congelar sus puntos acumulados en la ronda actual cuando decide no arriesgarse a superar los 21. Muestra un mensaje confirmando el puntaje final del participante e invoca a `pasaturno()` para ceder la tirada al siguiente jugador o a la computadora.
- **Elección Doble As (`#btnSumar2 / #btnSumar14`):** Permite resolver la jugada especial cuando salen dos Ases (1 y 1). Mediante la función `aplicarEleccionDobleAs(puntosElegidos)`, oculta la ventana modal, suma 2 o 14

puntos según la decisión del usuario y reanuda la posibilidad de tirar o plantarse si no se superaron los 21 puntos.

- **Reiniciar Partida (#jugarDeNuevo):** Permite abortar el juego en curso en cualquier momento mediante un evento **click**. Solicita confirmación con **confirm()**, oculta el tablero (**#pasoJuego.hidden = true**), restablece las instrucciones iniciales y reinicia las variables globales de estado (**listaNombres**, **victoriasGlobales**, **acumuladoRonda**).

Ciclo del Turno

- Ciclo de turno **Gestión del Turno (iniciarTurno())**: Muestra en pantalla el número de ronda y el participante activo, e inicializa la visualización de puntajes. Si le corresponde a la Computadora y la partida no ha terminado, deshabilita los botones de acción (**tirarDados y plantarse**) y ejecuta automáticamente **turnoComputadora()** tras 1 segundo mediante **setTimeout()**.
- **Renderizado de Resultados (renderizarImágenesDados() y aplicarPuntosTiro())**: Genera dinámicamente el código HTML para mostrar las imágenes de los dados obtenidos (**img/dados/{dado}.png**). Actualiza el texto con los puntos de la tirada y refresca en tiempo real la etiqueta **#puntajeActual** dentro del DOM.
- **Pase de Turno (pasaturno())**: Incrementa el índice del jugador activo (**jugadorActual++**). Si ya tiraron todos los participantes de la lista (**jugadorActual >= listaNombres.length**), invoca a **evaluarFinDeRonda()**; en caso contrario, ejecuta **iniciarTurno()** para la siguiente persona o el Bot.
- **Turno Automatizado de la CPU (turnoComputadora())**: La computadora ejecuta lanzamientos continuos mediante un bucle **while** mientras su acumulado de ronda sea menor a 17 (**acumuladoRonda < 17**). Si obtiene un Doble As (1 y 1), elige automáticamente sumar 14 puntos si no supera los 21, o 2 puntos en caso contrario; al superar o igualar los 17 puntos se planta y pasa el turno.

Lógica de Negocio y Reglas Especiales

A. Cálculo de Puntuación (calcularPuntosTiro)

Reglas de la tirada con dos dados:

- **Pares Iguales (excluyendo 1 y 1):** $(\text{dado1} + \text{dado2}) * 2$.
- **Suma Estándar:** $\text{dado1} + \text{dado2}$.

B. Evento Especial: "Doble As" (1 y 1)

1. Al salir **dado1 === 1 && dado2 === 1**, el sistema pausa el turno habilitando el modal **#modalDobleAs** (**modalDobleAs.hidden = false**).
2. Deshabilita los botones de tiro/plantarse (**tirarDados.disabled = true**).
3. El usuario decide mediante dos botones:
 - **btnSumar2:** Suma 2 puntos.
 - **btnSumar14:** Suma 14 puntos.
4. La función **aplicarEleccionDobleAs(puntos)** reanuda la lógica del juego.

Jugar contra la computadora (turnoComputadora)

- **Estrategia de tiro:** Mantiene tiradas continuas mediante un bucle **while** mientras **acumuladoRonda < 17**.
- **Evaluación de Doble As:** Si obtiene 1 y 1, evalúa si **acumulado + 14 <= 21**. Si cumple la condición, asigna 14 puntos; de lo contrario, asigna 2.
- **Cierre de turno:** Al alcanzar o superar los 17 puntos, detiene los lanzamientos. Si se pasa de 21, resetea a 0 y ejecuta **pasaturno()**.

3° Finalizar la partida

Una vez que alguien alcance las tres victorias el juego se finaliza y se muestran las victorias. El botón de reiniciar o el botón de ir a la página de los puntajes.

- **evaluarFinDeRonda():** Revisa las puntuaciones válidas de la ronda (≤ 21) para determinar al ganador del punto. Si un jugador alcanza o supera las 3 victorias (**victoriasGlobales[i] >= 3**), se activa la bandera **juegoTerminado = true**, se ocultan los botones **#tirarDados** y **#plantarse**, se muestra el mensaje de felicitaciones y se llama a **guardarPuntajesLocal()**.
- **guardarPuntajesLocal():** Recupera el historial existente desde el almacenamiento del navegador mediante **localStorage.getItem("dados_records")**. Transforma el texto JSON en un arreglo de objetos con **JSON.parse()**, añade los registros de cada jugador de la partida con sus victorias obtenidas y la etiqueta **juego: "21 con Dados"**, los ordena de mayor a menor (**b.victorias - a.victorias**) y vuelve a guardar el arreglo serializado con **JSON.stringify()**.
- **Reiniciar Partida (#jugarDeNuevo):** Permite volver a la pantalla inicial en cualquier momento tras confirmar mediante **confirm()**. Oculta el contenedor **#pasoJuego**, restablece las instrucciones iniciales (**#seccionInstrucciones**) y limpia las variables globales de estado (**listaNombres, victoriasGlobales, acumuladoRonda**).
- **Redirección a Puntajes:** Al igual que en el módulo de cartas, las victorias almacenadas en **localStorage** quedan listas para ser leídas y desplegadas dinámicamente en la tabla de posiciones general de la página "**puntaje.html**"

Estructura de Datos y Estado

El estado del juego se controla a través de variables globales que mantienen la sincronización de la interfaz y la lógica:

- **esContraCompu (boolean):** Controla la modalidad (Humano vs Bot o Humano vs Humano).
- **totalJugadores (number):** Cantidad de jugadores en la partida.
- **listaNombres (Array<string>):** Nombres de los participantes.
- **jugadorActual (number):** Índice del jugador activo en la ronda actual.
- **acumuladoRonda (Array<number>):** Puntos acumulados en la ronda activa por cada jugador
- **victoriasGlobales (Array<number>):** Victorias obtenidas a lo largo de las rondas.
- **ronda (number):** Contador incremental de rondas.

- **juegoTerminado (boolean):** Previene lanzamientos cuando finaliza la partida completa.
- **esperandoEleccionDobleAs (boolean):** Detiene la ejecución mientras se espera la decisión del modal interactivo del Doble As.

2.3 Juego de preguntas

Cinefilia

Este juego permite jugar de dos modalidades distintas: jugar solo o multijugador.

Instrucciones del juego:

Objetivo del Juego

Demostrar el conocimiento cinematográfico acumulado identificando afiches de películas en el menor tiempo posible. El participante que sume la mayor cantidad de puntos al cabo de 5 rondas se corona como el ganador de la partida.

Mecánica de Juego y Puntuación

En cada turno se presenta el afiche de una película con un efecto de desenfoque e inicia una cuenta regresiva de 15 segundos:

- **Desenfoque progresivo:** Cada 3 segundos la imagen reduce su nivel de desenfoque, volviéndose gradualmente más nítida a medida que el tiempo transcurre.
- **Acierto Temprano:** Si el participante selecciona la opción correcta, obtiene un puntaje base de 100 puntos más una bonificación proporcional a la nitidez pendiente (a mayor desenfoque al momento de responder, mayor puntaje obtenido).
- **Error o Tiempo Agotado:** Si el participante selecciona una opción incorrecta o el temporizador llega a 0, suma 0 puntos en la ronda y se revela la respuesta correcta en pantalla.

Mecanica del turno

El usuario interactúa directamente con la interfaz seleccionando una de las alternativas:

- **Opciones Múltiples:** En cada ronda se despliegan 4 botones con títulos de películas.
- **Selección:** Al presionar un botón, el temporizador se detiene, la imagen se revela por completo y se valida el resultado.
- **Transición:** Tras una breve pausa de 2 segundos, el sistema avanza automáticamente a la siguiente ronda o al turno del siguiente jugador.

Sistema de Puntuación y Rondas

- **Rondas por Participante:** Cada usuario juega 5 rondas consecutivas para registrar su puntaje total acumulado.
- **Ranking de Partida:** Al finalizar los turnos de todos los participantes, se comparan los puntajes acumulados y se despliega la tabla de posiciones finales.
- **Historial de Récores:** Los puntajes obtenidos se registran en el almacenamiento local para actualizar el ranking histórico general de la aplicación.

1° Captar datos:

Configuración e Inicialización

- Uso de **API Externa:** Se conecta de forma asíncrona mediante `fetch()` a la API de The Movie Database (TMDb)
(<https://api.themoviedb.org/3>)(<https://api.themoviedb.org/3>)).

Obtención de Películas (`obtenerPelículas`):

- Realiza una petición **GET** a la ruta `/movie/popular` pasando una página generada de forma aleatoria (`Math.floor(Math.random() * 5) + 1`).
- Filtra las películas para descartar aquellas que no poseen póster (`poster_path !== null`).

Ingreso de Participantes (Flujo Secuencial)

1. **Captura de Cantidad:** Se captura y valida la cantidad total de jugadores desde un campo de tipo `<input type="number">` (`#cantidadParticipantes`). Se valida que sea un número válido y mayor o igual a 1.
2. **Captura Secuencial de Nombres:** Los nombres se ingresan uno por uno mediante el botón `#ingresarNombre`.
3. **Manejo de Estado del Formulario:** A medida que se cargan los datos, los campos del formulario se deshabilitan (`disabled = true`) o limpian para evitar modificaciones inconsistentes durante el proceso.

2° Iniciar el Juego:

Carga de Turnos y Rondas (`cargarTurno`)

- Gestión de Participantes: Identifica al jugador activo según el valor de `indiceTurnoActual`.
- Generación Dinámica de Preguntas:
 - Mezcla de forma aleatoria el listado `películasCargadas` usando `.sort(() => 0.5 - Math.random())`.
 - Selecciona un subconjunto de 4 películas (`.slice(0, 4)`).

- Define la primera película (**opciones[0]**) como la película correcta (**peliculaCorrecta**).
 - Vuelve a desordenar las 4 opciones para renderizarlas como botones en el DOM sin una posición fija.
- Manejo Visual de la Película:
 - Carga la imagen del póster desde el CDN de TMDB (**https://image.tmdb.org/t/p/w500**).
 - Inicia con un nivel máximo de desenfoque CSS (**blur(15px)**).

Interacciones con la mesa / Temporizador

Temporizador y Revelación Progresiva (**iniciarTemporizador**)

- **Intervalo de Tiempo:** Se ejecuta una cuenta regresiva de 15 segundos mediante **setInterval()**.
- **Efecto de Desdesenfoque:** Cada 3 segundos transcurridos (**segundosRestantes % 3 === 0**), el valor de desenfoque disminuye en 3 unidades (**desenfoqueActual -= 3**), actualizando dinámicamente el estilo en el DOM (**imgPoster.style.filter = blur(...)**).

Respuestas e Interacción (**validarRespuesta**)

- **Limpieza de Intervalo:** Al responder o agotarse el tiempo, se detiene el temporizador con **clearInterval()**.
- **Cálculo Dinámico de Puntaje:**
 - **Respuesta Correcta:** Otorga un puntaje base de 100 puntos más un bono por velocidad basado en el desenfoque restante (**100 + (desenfoqueActual * 20)**). El botón seleccionado recibe la clase **CSS .correcta**.
 - **Respuesta Incorrecta o Tiempo Agotado:** El botón presionado recibe la clase **.incorrecta** y se invoca a **resaltarCorrecta()** para revelar visualmente cuál era la respuesta acertada.
 - **Revelado:** La imagen recupera inmediatamente nitidez total (**blur(0px)**).

3°Finalizar la partida:

Conclusión de Rondas y Transición (**avanzarFlujo**)

- Cada jugador disputa un ciclo de hasta **MAX Rondas** (5 rondas).
- Al completar sus 5 rondas, la partida pasa automáticamente al siguiente participante reiniciando su contador de rondas a 1.
- Una vez que todos los participantes completaron sus turnos, se muestra la pantalla final (**mostrarPantallaFinal**).

Pantalla Final y Persistencia (**guardarRecordsPartida / renderizarRecordsLocales**)

1. **Ranking de la Partida:** Renderiza los puntajes del juego recién concluido ordenados de mayor a menor.
2. **Web Storage (localStorage):**
 - Recupera los registros previos almacenados bajo la clave "cinefilia_records".
 - Deserializa los datos con `JSON.parse()`, añade las nuevas puntuaciones acumuladas y los vuelve a guardar serializados con `JSON.stringify()`.

Estructura de Datos y Estado Global

El estado del juego se controla a través de variables globales que mantienen la sincronización de la interfaz y la lógica de la trivia:

- **totalJugadores (number):** Cantidad de participantes ingresados para la partida actual (**totalJugadores = 1**).
- **listaNombres (Array):** Listado de los nombres ingresados por los usuarios (**listaNombres = []**).
- **jugadores (Array):** Estructura de objetos con el nombre y los puntos acumulados por cada participante (**[{ nombre: "...", puntos: 0 }]**).
- **indiceTurnoActual (number):** Índice del participante activo en el turno correspondiente (**indiceTurnoActual = 0**).
- **MAX Rondas (number):** Límite máximo de rondas por jugador (**MAX_RONDAS = 5**).
- **rondaActual (number):** Contador incremental de la ronda en curso para el jugador activo (**rondaActual = 1**).
- **películasCargadas (Array):** Almacén con las películas obtenidas desde la API de TMDB (**películasCargadas = []**).
- **películaCorrecta (Object | null):** Película seleccionada como la opción acertada en el turno actual (**películaCorrecta = null**).
- **timerInterval (number | null):** Referencia al temporizador de la cuenta regresiva (**timerInterval = null**).
- **desenfoqueActual (number):** Nivel del efecto CSS **blur** aplicado a la imagen del póster en el turno actual (**desenfoqueActual = 15**).
- **segundosRestantes (number):** Segundos restantes en el temporizador antes de finalizar el turno (**segundosRestantes = 15**).

API

Proveedor: The Movie Database (TMDB).

- **Endpoint Utilizado:** GET /movie/popular
- **Parámetros:**
 - **api_key:** Clave pública de autenticación.
 - **language:** Idioma de los datos (**es-ES**).
 - **page:** Número de página aleatorio entre 1 y 5 para variabilidad de catálogo.

3. Testing y experiencia del usuario.

1. Testing.

Caso de prueba: Visibilidad de la pantalla final y botón de reinicio

ID: CP-01

Funcionalidad: Pantalla final de la partida

Tipo de prueba: Prueba funcional / prueba de usabilidad

Objetivo: Verificar que la pantalla de resultados y el botón Jugar Nueva Partida permanezcan ocultos al iniciar el juego y aparezcan únicamente al finalizar la partida.

Pasos de prueba:

1. Abrir la página del juego.
2. Observar la pantalla antes de ingresar los participantes.
3. Comenzar una partida.
4. Completar todas las rondas.
5. Observar la pantalla al finalizar la partida.
6. Presionar el botón Jugar Nueva Partida.
7. Comprobar nuevamente el estado de la pantalla.

Resultado esperado:

Al iniciar la página, la sección de resultados, la pantalla final y el botón reiniciar deben permanecer ocultos y al finalizar la partida, ambos elementos deben hacerse visibles. Luego al presionar Jugar Nueva Partida, ambos elementos deben volver a ocultarse.

Resultado obtenido inicialmente:

La pantalla final y el botón Jugar Nueva Partida aparecían visibles desde el inicio de la página, antes de finalizar una partida.

2. Debugging.

Para investigar el problema se revisaron primero el HTML y el JavaScript.

En el HTML, ambos elementos tenían correctamente el atributo hidden.

`<section id="pantalla-final" hidden>` y `<button id="btn-reiniciar" type="button" hidden>`

Jugar Nueva Partida

</button>.

También se revisó el JavaScript. Allí se establecía correctamente que, al finalizar la partida, los elementos debían mostrarse:

```
pantallaFinal.hidden = false;
```

```
btnReiniciar.hidden = false;
```

Y al iniciar una nueva partida debían ocultarse nuevamente:

```
pantallaFinal.hidden = true;
```

```
btnReiniciar.hidden = true;
```

Por lo tanto, el problema no estaba en el JavaScript ni en el HTML.

Finalmente se revisó el CSS y se encontró la causa:

```
#pantalla-final {
```

```
    display: flex;
```

```
}
```

y

```
#btn-reiniciar {
```

```
    display: block;
```

```
}
```

Estas reglas establecen explícitamente que ambos elementos debían tener un valor de display visible. Esto entraba en conflicto con el comportamiento esperado del atributo hidden.

Solución implementada

Se agregó al final del CSS una regla específica para los elementos que poseen el atributo hidden.

```
[hidden] {
```

```
    display: none !important;
```

```
}
```

De esta manera, cuando los elementos tienen hidden, CSS establece explícitamente que no se ejecute el display mientras que cuando JavaScript modifica: **elemento.hidden = false;** el elemento vuelve a mostrarse utilizando los estilos definidos originalmente.

Resultado después del debugging.

Se volvió a ejecutar el caso de prueba desde el comienzo y la pagina funciono como lo esperado:

- La pantalla final permanece oculta al cargar la página.
- El botón "Jugar Nueva Partida" permanece oculto durante el juego.
- Ambos aparecen al finalizar la partida.
- Al seleccionar "Jugar Nueva Partida" vuelven a ocultarse.

3. Revisión de UX.

Se realizó un test de usabilidad con una persona que juega videojuegos desde su infancia y pasa varias horas del día en plataformas de juegos digitales.

Datos de la persona:



Nombre Martina

Age: 25

Education: universitaria

Hometown: Florencio Varela

Family: Soltera

Occupation: Customer service

"Juego en la computadora desde que tengo 10 años. Cuando era chica jugaba en paginas, ahora uso torrent o steam"

Objetivos

- Entretenimiento rapido y sin tener que pensar demasiado.

Frustraciones

- interfaces que sean complicadas y requieran de muchos pasos para hacer una tarea sencilla.

Hace mucho no veo paginas de juegos en linea, siempre que nos juntamos con mis amigos jugamos juegos de cartas y cuando eramos más chicos haciamos torneos de truco online.

La tarea a realizar fue entrar en el sitio y utilizarlo sin instrucciones externas

Se propusieron 10 casos de prueba a verificar mientras la persona navegaba en el sitio web desde github pages.

ID	Caso de prueba.	Acción del usuario.	Resultado.
CP-01	Acceso al inicio.	Revisa la página principal.	El usuario identifica rápidamente de que se trata el sitio, logra distinguir de qué se trata cada juego a través de las imágenes y entiende que

			puede hacer dentro de la pagina.
CP-02	Comprensión del menú de navegación.	Observa las opciones del navegador sin hacer click.	El usuario comprende para que sirve cada opción solo con leer el nombre utilizado.
CP-03	Inicio de una partida	Ingresa sus datos correctamente en los campos y comienza una partida de 21.	El usuario comprende que debe apretar el botón iniciar partida para comenzar una partida, comprende las opciones del menú de entrada y coloca satisfactoriamente sus datos y comienza una partida.
CP-04	Datos invalidos	Introduce datos erróneos en la partida de preguntas.	El usuario lee la advertencia que le proporciona el juego, comprende su error y vuelve a colocar los datos correctamente. Cuando intenta interactuar con el cuadro para ingresar el nombre antes de poner el número de participantes para repetir una partida se frustra y encuentra el proceso repetitivo y fastidioso.
CP-05	Comprensión de instrucciones.	Lee las instrucciones del juego.	El usuario se queja de que las instrucciones son muy largas y las lee por encima, igualmente puede avanzar y jugar el juego sin problemas. También remarca que hay problemas de puntuación en la pagina del juego de preguntas.
CP-06	Juego de preguntas - Tiempo.	Deja transcurrir el tiempo disponible en una pregunta.	El sistema marca la respuesta correcta y sigue con el flujo del juego. La próxima pregunta no aparece de inmediato, tarda un poco en aparecer.
CP-07	Finalización de la partida.	Completar una partida.	Al usuario le resulta confuso que aparezcan

			los datos de su turno antes del fin del juego. Puede identificar los puntajes obtenidos y que es el ganador de la partida.
CP-08	Reiniciar partida.	Hace click en el botón jugar nueva partida.	El juego vuelve a un estado inicial, los formularios no conservan datos que deberían borrarse.
CP-09	Navegación a puntajes.	Buscar y acceder a la página de puntajes.	El usuario encuentra fácilmente sus resultados y comprende la disposición.
CP-10	Uso sin conocimientos previos de la página.	Navegación de la página sin problemas.	El usuario puede iniciar y completar un juego sin recibir instrucciones externas.

Notas extra dejadas por el usuario evaluando los problemas:

Juego de preguntas:

- Se repiten las mismas películas en las rondas de juego de preguntas, las elija bien o mal.
- Cuando terminas una ronda y presionas jugar nueva partida aparecen dos cuadros uno para elegir el número de participantes. No debería estar ahí si soy la misma persona que vuelve a repetir el juego.

Juego de cartas:

- Después del turno de la computadora el texto sigue siendo "la computadora está pensando.." cuando debería ser selecciona una carta de tu mano para jugar, el texto cambia cuando seleccionas una carta.
- A pesar de que no se jugar a la casita robada el juego se me hizo fácil.

4. Revisión con IA.

Se generó un prompt a chatGPT para que detecte riesgos o problemas y se le adjunto el zip del proyecto.

La ia detectó los siguientes problemas:

Informe de revisión UX/UI y riesgos funcionales

Revisé el archivo ZIP que compartiste del proyecto TP-Informatica-General-n-1. Esto me permite analizar el código HTML, CSS y JavaScript, aunque no equivale a ejecutar la página publicada en un navegador ni a completar partidas reales.

A continuación separo los problemas que pude identificar directamente en el código de los aspectos que todavía necesitarían una prueba interactiva. Las prioridades indican el impacto potencial para el usuario, no una medición de frecuencia.

1. Problemas detectados en el código

Hallazgo 1. El historial local de Cinefilia Trivia no muestra los registros

Alta

Archivo: `js/preguntas.js`

La función `renderizarRecordsLocales()` limpia la lista y ordena los registros, pero no crea ni agrega elementos para mostrarlos. Por eso, cuando existen registros, el historial local puede quedar vacío aunque los datos estén guardados.

Cómo comprobarlo: completar una partida, llegar a la pantalla final y observar «Historial de Récords Locales».

Riesgo UX: el usuario puede creer que sus resultados no se guardaron.

Corrección sugerida: recorrer `records` y agregar un elemento a `listaRecords` por cada registro, después de ordenarlos.

Hallazgo 2. El formulario de nombres no se oculta al comenzar la partida

Alta

Archivo: `js/preguntas.js`

Después de ingresar el último nombre, el código inicia la carga de películas, pero no oculta explícitamente `ingresoDatos3`. La función que inicia la partida tampoco lo oculta.

Cómo comprobarlo: iniciar una partida, ingresar la cantidad de participantes y completar sus nombres.

Riesgo UX: el formulario podría continuar visible junto al juego y generar confusión sobre qué acción corresponde realizar.

Corrección sugerida: ocultar el formulario de nombres al completar el registro de participantes y antes de mostrar el tablero.

Hallazgo 3. El reinicio de Cinefilia Trivia muestra dos formularios

Media

Archivo: `js/preguntas.js`

Al pulsar «Jugar Nueva Partida», el código asigna `false` a `hidden` tanto para `ingresoDatos2` como para `ingresoDatos3`. Al mismo tiempo, deja deshabilitados el campo y el botón para ingresar nombres.

Cómo comprobarlo: finalizar una partida y pulsar «Jugar Nueva Partida».

Riesgo UX: el usuario puede ver simultáneamente la solicitud de cantidad y el formulario de nombres deshabilitado, lo que hace menos claro el orden de los pasos.

Corrección sugerida: mostrar únicamente el formulario de cantidad al reiniciar y habilitar el de nombres cuando corresponda.

Hallazgo 4. Error tipográfico en un botón de la página de inicio

Baja

Archivo: `index.html`, línea 401.

El botón que lleva al juego de cartas muestra `PILAY` en lugar de `PLAY`.

Riesgo UX: transmite una impresión de falta de revisión y reduce la consistencia de la interfaz.

Corrección sugerida: reemplazar el texto por `PLAY` o usar una etiqueta en español, como «Jugar».

Hallazgo 5. Etiquetas de formulario incompletas en el juego de cartas

Media

Archivo: `cartas.html`, líneas 69–79.

Las etiquetas de cantidad y nombre no están asociadas explícitamente con sus campos mediante `for` e `id`. En comparación, el formulario equivalente de dados sí vincula sus etiquetas a los campos.

Riesgo UX y accesibilidad: puede dificultar la identificación de los campos mediante tecnologías de asistencia y hacer menos cómoda su selección.

Corrección sugerida: agregar `for="cantidadParticipantes"` y `for="nombreJugador"` a las etiquetas correspondientes.

2. Riesgos adicionales que conviene probar

Estos puntos son riesgos identificados a partir del código, pero necesitan pruebas de interacción para confirmar su impacto real.

ID	Riesgo	Prueba recomendada
R-06	Cantidad excesiva de participantes. Los campos de cantidad de los juegos no establecen un máximo en el HTML; las validaciones JavaScript tampoco establecen un límite superior en los fragmentos revisados.	Introducir una cantidad muy grande y comprobar si el usuario puede avanzar, si la interfaz sigue siendo manejable y si existe alguna limitación clara.
R-07	Fallo de conexión con la API de películas. El código contempla errores de conexión y respuestas fallidas, pero no se observa un mecanismo explícito de reintento en ese flujo.	Simular una desconexión o una respuesta fallida y comprobar si el usuario puede recuperarse sin recargar toda la página.
R-08	Borrado de todos los puntajes. La sección general tiene una acción para borrar los registros de los tres juegos.	Pulsar «Reiniciar Puntajes», cancelar la confirmación y verificar que no se borren los datos; luego aceptar y verificar que se actualice la pantalla.
R-09	Accesibilidad mediante teclado. El código incluye botones, enlaces y campos de formulario, pero el análisis estático no permite confirmar si todos tienen un foco visible y un orden de navegación adecuado.	Recorrer todo el sitio usando Tab , Shift + Tab , Enter y la barra espaciadora, sin utilizar el mouse.
R-10	Adaptación a dispositivos móviles. La hoja de estilos incluye una regla @media (max-width: 600px) , pero eso no garantiza que todos los juegos se adapten correctamente.	Probar en un teléfono y verificar que no haya desplazamiento horizontal, botones cortados, textos superpuestos o cartas demasiado pequeñas.

3. Casos de prueba prioritarios para una sesión de usuario

Lista de verificación

CP-01 — Completar una partida de Cinefilia Trivia y revisar el historial local.

Resultado esperado: Los resultados aparecen en el historial de la pantalla final.

Prioridad: Alta

CP-02 — Reiniciar una partida de Cinefilia Trivia.

Resultado esperado: Solo se muestra el paso correspondiente a ingresar la cantidad de participantes.

Prioridad: Alta

CP-03 — Ingresar los nombres de los participantes y comenzar el juego.

Resultado esperado: El formulario desaparece y queda visible el tablero.

Prioridad: Alta

CP-04 — Ingresar una cantidad excesiva de participantes.

Resultado esperado: Se aplica un límite razonable y se informa claramente si se supera.

Prioridad: Media

CP-05 — Intentar jugar cuando la API no responde.

Resultado esperado: El sistema informa el problema y ofrece una forma clara de recuperarse.

Prioridad: Alta

CP-06 — Navegar por todos los juegos usando solo el teclado.

Resultado esperado: Los controles tienen foco visible y pueden activarse sin mouse.

Prioridad: Media

CP-07 — Borrar los puntajes y cancelar la confirmación.

Resultado esperado: Los registros permanecen intactos.

Prioridad: Media

CP-08 — Usar cada juego desde un teléfono.

Resultado esperado: Los elementos se ven completos y los controles se pueden utilizar cómodamente.

Prioridad: Media

4. Alcance y conclusión

La revisión del ZIP permitió detectar cinco problemas concretos de implementación o interfaz y señalar riesgos adicionales que merecen pruebas específicas. No puedo afirmar

que todos se manifiesten de la misma manera en la versión publicada sin ejecutar el sitio en un navegador.

Además, comprobé la sintaxis de los archivos JavaScript con una herramienta de análisis sintáctico: los cinco archivos pasaron esa comprobación. Eso no demuestra que sus funciones operen correctamente durante una partida.

Mi recomendación: comenzar por el historial de Cinefilia Trivia, el flujo de inicio y reinicio, y después probar el manejo de errores de la API. Son los escenarios con mayor potencial de afectar la continuidad de uso o la confianza del usuario en sus resultados.

5. Priorización.

Se decidió priorizar el caso en que la ia y la prueba de usuario coincidieron: el hallazgo 2 que detectó la ia y el usuario dejó asentado como molesto en las notas.

6. Mejora.

1. El problema detectado

Al presionar el botón de **"Jugar Nueva Partida"**, el juego intentaba volver al estado inicial, pero **el recuadro de fin de partida y el botón de reinicio no desaparecían de la pantalla.**

El código del evento presentaba esta estructura:

```
btnReiniciar.addEventListener("click", () => {  
  
  pantallaFinal.hidden = true;  
  
  btnReiniciar.hidden = true;  
  
  marcador.hidden = true;  
  
  pasoJuego.hidden = true;  
  
  ingresoDatos2.hidden = false;  
  
  ingresoDatos3.hidden = false; // <-- Error de flujo visual  
  
});
```

2. Causas del error

Identificamos la causa principal para este fallo:

Asignación errónea en el flujo: En el JS se estaba pasando a **ingresoDatos3.hidden = false**, mostrando el formulario para pedir nombres de forma prematura en lugar de mantenerlo oculto hasta definir la cantidad de participantes en **ingresoDatos2**.

3. Cómo lo resolvimos

Para solucionar el inconveniente aplicamos la siguiente solución:

Corrección de la lógica en JavaScript (preguntas.js)

Ajustamos el evento para ocultar **ingresoDatos3** y restaurar únicamente el primer paso de configuración:

```
btnReiniciar.addEventListener("click", () => {  
  
  // 1. Ocultar la pantalla de resultados y el botón  
  
  pantallaFinal.hidden = true;  
  
  btnReiniciar.hidden = true;  
  
  marcador.hidden = true;  
  
  pasoJuego.hidden = true;  
  
  
  // 2. Mostrar solo el primer paso (ingreso de cantidad) y ocultar el de nombres  
  
  ingresoDatos2.hidden = false;  
  
  ingresoDatos3.hidden = true; // Corregido a true  
  
  
  // 3. Restablecer controles y variables de estado  
  
  numeroParticipantes.value = "";  
  
  numeroParticipantes.disabled = false;  
  
  ingresarCantidad.disabled = false;  
  
  
  nombreJugador.value = "";  
  
  nombreJugador.disabled = true;  
  
  ingresarNombre.disabled = true;  
  
  
  listaNombres = [];  
  
  jugadores = [];
```

```
indiceTurnoActual = 0;

rondaActual = 1;

clearInterval(timerInterval);

});
```

7. Nueva prueba.

Se volvió a ejecutar el caso de prueba desde el comienzo y la pagina funciono como lo esperado:

- El formulario de ingresar nombres aparece oculto.
- El formulario de ingresar nombres se habilita luego de ingresar la cantidad de participantes.

4. Declaración de IA.

Se utilizaron herramientas de Inteligencia Artificial Generativa (Gemini, Claude y ChatGPT) como apoyo técnico, conceptual y creativo en distintas etapas del desarrollo de este trabajo práctico. A continuación, se detalla la contribución específica en cada área:

1. Concepción y Desarrollo de Juegos

Lógica y reglas de juego: Se consultó a la IA sobre la factibilidad técnica de las ideas iniciales para los distintos juegos.

- **Juego de Dados (21):** La IA asistió en la estructuración de la lógica interna de puntuación y rondas, además de redactar la guía de instrucciones para los usuarios.
- **Juego de Preguntas (Cinefilia):** Se utilizó la IA para proponer alternativas que cumplieran con el requisito de dificultad progresiva de la consigna. Sugirió la implementación del efecto de desenfoque (*blur*) dinámico con temporizador en los pósteres de las películas obtenidas mediante la API de TMDb, transformando la trivia en un desafío visual. Asimismo, se redactaron las instrucciones del juego.
- **Juego de cartas (Casita Robada):** La IA ayudó a redactar y sintetizar el cuerpo de instrucciones del juego.

2. Desarrollo de Código y Arquitectura Web

- **HTML5 Semántico y Accesibilidad:** Se consultó sobre las mejores prácticas de maquetación semántica (`<header>`, `<main>`, `<nav>`, `<footer>`, `<section>`) para estructurar de manera limpia las diferentes páginas del sitio (`index.html`, `preguntas.html`, `puntaje.html`, etc.).
- **Estilos y Layout con CSS:** Apoyo en el diseño de interfaces responsivas utilizando Flexbox, centrado de elementos, paleta de colores personalizada y reglas de diseño

adaptable (Box Model, unidades relativas en **rem** y diseño de capas/pestañas que no llegaron a utilizarse porque no estaban en los contenidos de la cátedra).

- **Lógica en JavaScript:** Asistencia en la implementación del consumo asíncrono de APIs externas (**fetch** con **async/await**), manipulación dinámica del DOM (**createElement**, **appendChild**, **querySelector**), control de temporizadores (**setInterval**, **clearInterval**) y gestión de persistencia de datos mediante la Web Storage API (**localStorage**).

3. Detección, Depuración de Errores y Refactorización (Debugging)

- **Resolución de conflictos en el DOM:** Diagnóstico y corrección de errores sintácticos y de especificidad en CSS (por ejemplo, conflictos de visibilidad entre la propiedad **display** y el atributo HTML **hidden**).
- **Sincronización de Estados y Variables:** Corrección en el flujo de ejecución de funciones asíncronas, limpieza de formularios al ingresar participantes y gestión del reinicio general de partidas y puntuaciones globales.
- **Estandarización de Código:** Asistencia en la unificación y mantenimiento del estado en páginas independientes (**puntaje.js** y **preguntas.js**).

4. Creación de Contenido e Imágenes

- **Generación de Prompts e Imágenes:** Se usó en la elaboración de prompts y selección de recursos gráficos visuales para acompañar la interfaz de la página.

5. Evaluación de Experiencia de Usuario (Diseño UX/UI)

- **Generación de casos de prueba para Test de UX:** La IA colaboró en la formulación de escenarios, tareas clave de usuario y casos de prueba para evaluar la usabilidad de la plataforma (navegación entre juegos, claridad en el ingreso de participantes, legibilidad de la pantalla de resultados y facilidad de lectura de los puntajes globales).

6. Estructuración y Redacción de Documentación

- **Organización del informe general:** La IA colaboró en la estructuración temática, jerarquización de contenidos y redacción formal de fragmentos de la documentación técnica del proyecto, garantizando coherencia en los conceptos presentados y adaptándolos a los requerimientos del trabajo práctico.